

Fakturama Image-to-Cash — Design Document

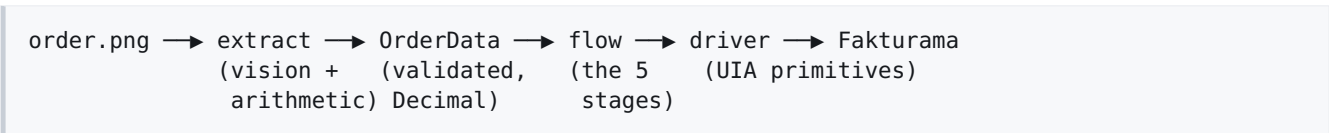
Ahmed Islam Farouk Abbas · TJM Labs take-home, Part 1 · one-page version: `DESIGN-BRIEF.md`

1. Problem

In	one order image
Out	a saved, verified Order and its linked Invoice
Resolved from Fakturama's own selectors	Debtor, Payment Method, VAT rate, every Product — created only when no exact match exists
Forbidden	hardcoded screen coordinates; any assumption about window size, theme or layout

The interesting part is not clicking. It is deciding, at each step, whether what I am looking at is the thing I meant — and stopping when I cannot tell.

2. Shape of the system



LAYER	PROPERTY	CONSEQUENCE
extract	pure — bytes in, dataclass out	no GUI, no Windows, no Fakturama; ~40% of the work ships before the VM exists
flow	the spec transcribed, stage for stage, step numbers in the comments	a reviewer can diff it against their own document
driver	the only module that knows what a window is	Fakturama 2.2 moves a button → one file changes

3. Control discovery and grounding

The obstacle

REALITY	EFFECT
SWT renders native Win32 widgets	UIA sees a real control tree, not one opaque canvas — this is why UIA is the right primary tool
SWT sets almost no <code>AutomationId</code>	most elements surface only a <code>Name</code> , derived from their label
<code>Name</code> comes from the label	locale-dependent; a German install breaks every name-based lookup
Some elements have no name at all	the two icons beside <code>Addresses</code> , where the spec is emphatic about the <i>upper</i> one, are distinguished by image and tooltip

A naive `find(name=...)` is therefore both fragile and ambiguous.

Four tiers, first unambiguous hit wins

TIER	METHOD	WHY IT EARNS ITS PLACE
T1	scoped tree search — resolve the active editor or dialog first, then search <i>within</i> it by <code>ControlType</code> + <code>Name</code> / <code>AutomationId</code>	scoping is what makes "Save" mean <i>this Order's</i> Save, and keeps the still-open Order tab from colliding with the Debtor editor opened on top of it

TIER	METHOD	WHY IT EARNS ITS PLACE
T2	anchor-relative walk — find a stable named anchor (<code>Addresses</code> , the <code>Items</code> header), then take the nth child of its sibling container	expresses "the upper icon, not the lower green +" as tree position rather than pixel position
T3	property disambiguation — <code>HelpText</code> , tooltip, <code>LegacyIAccessible.Description</code> , <code>IsEnabled</code>	separates structurally identical siblings
T4	vision tiebreak — screenshot the anchor element's own bounding rectangle , obtained from UIA at runtime, pass it to an LLM with the candidate list, click the element it names by its UIA handle	one unnamed toolbar icon cannot sink the whole flow

T2 is the direct answer to "no hardcoded coordinates." Ordering is a property of the widget hierarchy, not of the screen, and positions are read from the live tree at call time — so a resized, re-themed or differently-DPI'd window still resolves. T4 is not a coordinate either: the model decides *which* control, UIA supplies *where* it is, and nothing is authored by hand or persisted between runs.

I expect T4 to fire rarely, possibly never. It exists so that the failure mode I would otherwise most worry about cannot stop the run.

Selector registry and waiting

CONCERN	DECISION
Where selectors live	one dict: logical name → (tier, scope, name, anchor, index) . A version bump or a German install becomes a data edit, not a code change. Moves to YAML the moment a second locale is real.
How waiting works	no fixed delay ever stands in for a check. Every action is followed by a postcondition poll on the UIA tree with a timeout: "the New Order editor exists", "Cust.Ref. equals WEB-2026-0714-A17".
Why that matters	waiting on observed state rather than elapsed time is what makes a run reproducible on a cold machine and a warm one.

4. Image extraction

The supplied input is a clean, machine-rendered document — not a crumpled scan. That changes which tool is correct.

ROLE	CHOICE	REASON
Primary	LLM vision constrained to a JSON schema	it reads <i>table structure</i> — which column is <code>Disc.</code> and which is <code>VAT</code> , which value belongs to <code>BILLING</code> — rather than returning a bag of words whose geometry I then rebuild. Label→value pairing is exactly what flat OCR discards and exactly what this document is made of.
Fallback	local OCR — no account, no key	keeps a run alive when every API is absent or rate-limited
Geometry, always	local OCR	measured bounding boxes, deterministic. Asked for a column centre, a model drifted 12–15 points between two reads of the <i>same</i> table

Measured, not assumed: [docs/BENCHMARK.md](#) . The implementation is `rapidocr-onnxruntime` , not Tesseract — one pip package, no system binary, which matters when a reviewer has to reproduce a Windows VM. Tesseract has not been benchmarked here.

Trust nothing; validate arithmetically

The document is self-checking, so extraction correctness is *decidable without a human*.

IDENTITY	ON THE SUPPLIED IMAGE
<code>line_net == qty × unit_net × (1 - disc/100)</code>	<code>2×250×0.90 = 450.00</code> , <code>3×40 = 120.00</code>
<code>net_total == Σ line_net</code>	<code>570.00</code>
<code>vat_total == Σ (line_net × vat/100)</code>	<code>19% → 108.30</code>
<code>gross == net_total + vat_total</code>	<code>678.30</code>

All four hold. A violation means a misread digit: re-prompt once naming the failed identity, halt if it fails again. Stronger than running a second OCR engine and diffing strings, and it adds no dependency.

Its honest limit: it catches digit errors only where they break an identity. A misread SKU or a wrong city is invisible to arithmetic, and is caught later by Fakturama's own selector returning no exact match.

Normalization happens at the boundary: dates → ISO, money → Decimal (never float; 108.30 has no binary representation and this is a money path), percentages → int.

5. Verification model

The spec's real subject is verification, and it states two invariants worth naming.

INVARIANT	MEANING
Selection is the existence check	never query the database or a list view to decide whether a Debtor or Product exists — open the Order's own picker and search it. A record that exists but is not selectable <i>from the Order</i> is correctly treated as absent
Saving is confirmed by re-selection	after creating a record, the proof it persisted is that the <i>Order's</i> picker can now find it — round-tripping through the consumer rather than trusting the producer's own "saved" state

Postconditions, in order:

#	CHECKED
1	invoice/delivery addresses populated after Debtor selection
2	the line-price identity, after each item
3	Order totals against the image
4	the <code>Data > Documents</code> Order row — number, date, Cust.Ref., open state, total
5	the Invoice row, with the source Order still open at the same Cust.Ref. and total

One trap in the supplied data. Step 2.8 says to assign the Delivery role to the Main address *if billing and delivery are identical*. In this image they are not — Friedrichstrasse 88 / 10117 against Beusselstrasse 44 / 10553 — so an unconditional shortcut produces the wrong Debtor. The comparison is made on normalized address fields, never assumed either way.

What the application turned out to be is a separate document, [docs/FINDINGS.md](#). Everything in it was found by running against Fakturama 2.2.0 rather than by reading the spec: this is the design as argued beforehand, that is the correction.

6. Halting

Where	2.3, 2.10.2, 3.3, 3.5, 3.12, 5.2
Shape they share	more than one exact candidate, or a candidate whose properties conflict with the source
What is raised	one <code>AmbiguityHalt</code> , carrying the logical control, the query issued, the candidates observed, and a screenshot
Result	non-zero exit and an artifact a human can act on

Guessing between two Debtors is the one failure this system must never produce — a wrong invoice sent to a real company is worse than no invoice.

Idempotency falls out of this. Every stage is check → create → re-select, so a second run finds the master data present and skips creation. Re-running after a crash is safe up to the Order itself.

7. Tradeoffs

DECISION	BOUGHT	PAID
UIA first, vision only as tiebreak	fast, deterministic, no per-step API cost	requires Windows; SWT's thin <code>AutomationId</code> coverage forces T2
Anchor-relative over coordinates	survives resize, DPI and theme changes	one indirection; breaks if the widget hierarchy is restructured
LLM vision over OCR alone	table structure and label→value pairing for free	non-deterministic output, mitigated by the arithmetic gate and by letting OCR own geometry
Arithmetic self-check over dual-OCR	decidable correctness, zero extra dependencies	blind to errors that preserve the identities
Selector dict	locale/version changes are data edits	slight indirection when reading the flow
Halt on ambiguity	no silently wrong financial records	the interesting cases need a human
Order tab held open throughout	matches the spec; no re-navigation cost	longer-lived UI state to keep straight

8. Known ceiling

LIMIT	DETAIL
One locale, one version	English, Fakturama 2.2.0. Both are selector-dict data, not code.
Single-window assumption	an unanticipated modal blocks the postcondition poll until timeout, then halts — safe, but not recovered from
Vision tier cost	one API call per unresolved control. If UIA coverage is as good as expected, that tier stays cold.
Source image	one page, upright. Multi-page or rotated input is out of scope.